

• O que é um engenheiro de software:

- Projetar.

- Construir.

- Manter.

- Escrever código.

- Realizar testes.

- Lidar com stakeholders.

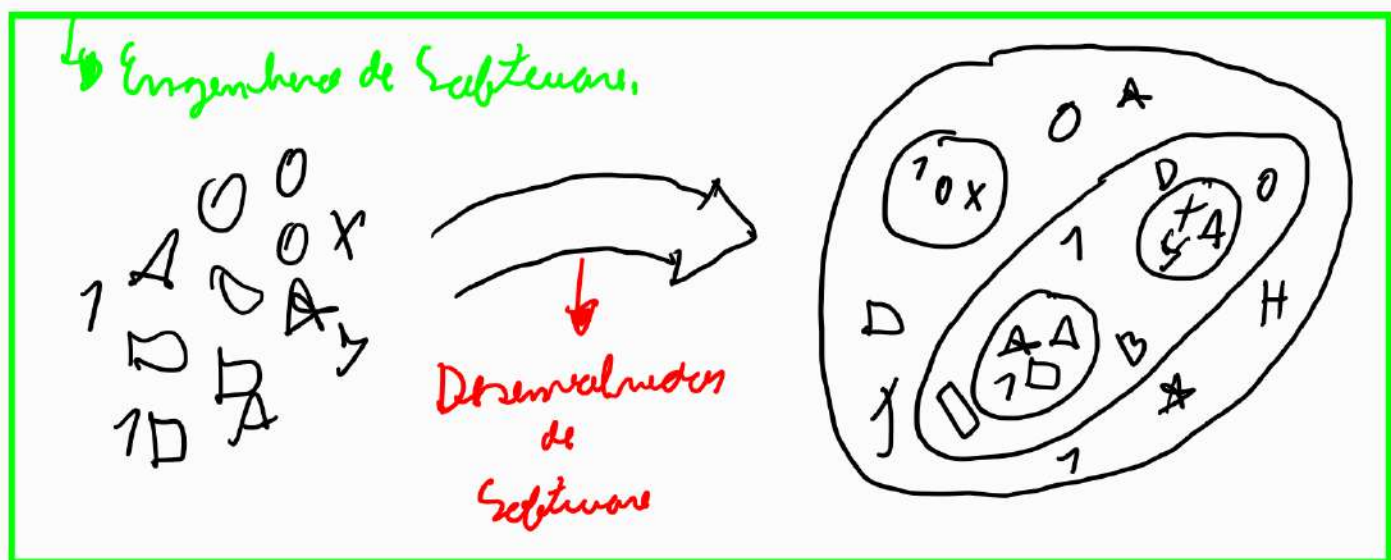
- Lidar com fornecedores terceiros.

- Tratar com especialistas.

- Lidar com outros membros.

Software engineers > Software developers

- Para mim o que é um engenheiro de software:
 - Assim como em outros engenharias, o engenheiro de software é capaz de unir os diversos elementos avulsos naturais em um conjunto dos mesmos, porém harmonizados e com funções específicas exercitadas a vontade humana por meio de ferramentas e da criatividade.
 - Um desenvolvedor de software diferentemente do engenheiro é quem transporta os elementos avulsos para os devidos lugares, também através de ferramentas, porém quem define, pesquisa a estrutura os lugares é o engenheiro.



• SDLC:

- Abordagem que nos ajuda para lidar com a crise de software (1960-1980),
- Tem fases claras e definidas (notas bem definidas)
- Cada membro tem um papel bem definido.
- Pode ser usado iterativamente, permitindo mudanças nos requisitos.
- Oferece maior comunicação entre a equipe e os stakeholders.

• Ciclo SDL:

7º Planejamento:

- Reunir requisitos.

- Analisar requisitos.

- Documentar requisitos. (SRS)

SRS = Deve ser aprovada pela equipe e pelos stakeholders.

- Priorizar os requisitos.

- Estimar custos (mão de obra, material, tempo)

- Identificar equipes e suas funções.

- Construção de um pequeno protótipo para validação dos stakeholders. Comumente apenas uma "apresentação" de como seria a interface do design.

2º Design:

- Desenvolvimento da arquitetura com base no SRS.
- A arquitetura deve ser aprovada por todos, a equipe e os stakeholders.
- Documentação do design (com a arquitetura).
- Pode também ser criado um protótipo demonstrando partes de funcionalidade, ou um todo.

3º Development:

- Codigos com base no documento de design.
- Determinar e atribuir tarefas de codificação para a equipe, com base na documentação.
- Uso de diferentes ferramentas de programação.

4º Testing:

- Lem todos os códigos prontos, os mesmos devem ser exaustivamente testados para garantir que estão, seguros e atendendo aos requisitos na SRS.
- Testes podem ser manuais, automatizados, ou híbridos.
- Os "bugs" identificados devem ser relatados, construídos e corrigidos, e códigos e testes novamente até que o software esteja estável.
- Testes comuns: Unit, Integration, System, Acceptance.

5º Deployment:

- Enviada para produção.

- Normalmente dividida em etapas para maior segurança.

- Primeiro pode ser lançado para uma plataforma de teste de integração de código: V.A.T.

- Lançamento gradual para grupos específicos.

6º Manutenção:

- Após produção deve ser feito um acompanhamento contínuo a cerca dos

"bugs", interbase, requisitos, melhoria de código

O software deve ser mantido atualizado

a cerca de seus problemas ou atualizações.

• Requisitos:

- Funcional: Funções do software.
- Externa e user interface: Comportamento do software em relação a entidades externas (usuário, hardware, software).
- System features: Requisitos para que o sistema funcione.
- Nonfunctional: Segurança, desempenho e qualidade.

• Design:

↳ Define os padrões.

- É o processo de transformar requisitos em uma estrutura implementável usando código.
- Transforma os requisitos em uma solução de software.
- Incorpora orientações sobre funções, desempenho, segurança e características da plataforma do sistema.
- Comunica os regras de negócios e a lógica do aplicativo.
- Define a interface que é como os aplicativos comunicam entre si com o banco de dados.

- Coding quality:
 - Maintainability:
 - Fácil de ler e manter.
 - Readability:
 - Fácil de compreender.
 - Bem documentado.
 - Testability:
 - Possível de testar.
 - Security:
 - Seguro
 - Eficiente

* Devem seguir os padrões, convenções e estilos estipulados.

* Devem utilizar lógicas mais difíceis de compreender.

• Testing:

- Unit Testing:

- Testa o menor componente possível do código que pode ser isolado do resto do sistema!

- Integration Testing:

- Testa vários códigos que se comunicam entre si, testa se os códigos interagem quando em conjunto funcionam de maneira correta.

- System Testing:

- Testa o produto totalmente ou quase concluído como um todo, teste robusto que verifica se tudo está funcionando.

- UAT (User acceptance testing) ou Beta:

- Testa a utilização do usuário.

* Os Testes podem ser divididos em três categorias:

- Funcional, não funcional e regressão.

● Releases: (lançamentos)

- Comumente (um padrão) não são três conjuntos:

- Alpha:

- lançada para poucas pessoas (stakeholders)
- Contem erros.
- Pode não ter todos os recursos, mas tem boa parte.
- Ainda ocorrem muitos mudanças.

- Beta:

- Mais pessoas tem acesso.
- A intenção é testar e reportar em condições reais.
- Deve conter todos os requisitos funcionais.

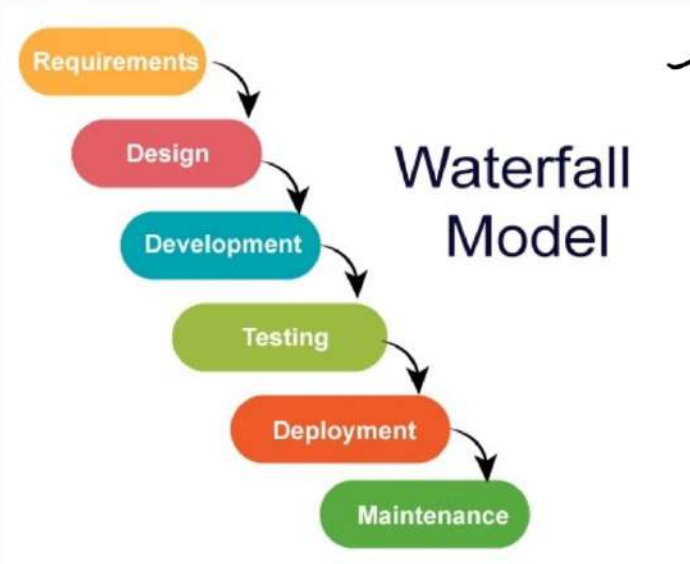
- General Availability:

- lançada a todos os usuários.
- Deve estar funcionando e com todos os problemas desativados nos outros versões corrigidos.

• Development methodologies:

* Waterfall (cascata):

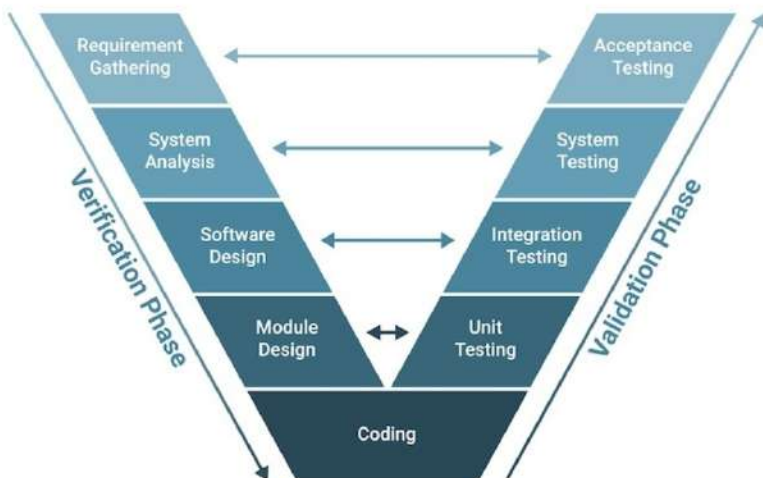
- Sequential, uma fase só começa depois que a outra termina.



• Demonstração do funcionamento.

* V-shape:

- É dividida por verificação e validação.



• Demonstração
onde cada lado do V
é referente ao parcer,
sendo 4 de cada lado.

* Agile:

- Abordagem iterativa de desenvolvimento
as equipes trabalham com pequenos sprints (ciclos)
que duram um curto período de uma a quatro semanas.
Testes unitários acontecem em cada sprint!
A estratégia final de uma sprint: o feedback, onde
os códigos prontos são apresentados aos stakeholders
em uma demonstração de sprint.
Após alguns sprints um MVP é criado para
apresentar os stakeholders.
Metodologia mais usada no mercado atual.



→ Demonstração
onde cada sprint
segue este ciclo
de maneira em um
curto período.

• Principios do Ágil:

Manifesto Ágil



• Testes:

- Funcional testing:

- Testa a funcionalidade, se está com a lógica adequada de acordo com os requisitos.

- Non-functional testing:

- Testa o código em diversos ambientes diferentes para garantir seu desempenho, segurança, recuperação, tudo aquilo relacionado ao "background" do código.

- Regression Testing:

- Testa após uma atualização ou correção de bugs se tudo ainda está funcionando como deveria e se não houve efeito a nova atualização.

• Cargos na programação:

* Project manager / Scrum master:

- Project manager:

- Planeja, Agenda, organiza.
- Aloca pessoas e recursos.
- Executa o plano de trabalho.
- Comunica com todo time.
- Cargo de altas responsabilidades.

- Scrum master (Agile):

(Agile prioriza a comunicação)

- Responsável por facilitar a comunicação da equipe.
- Responsável por gerenciar o entendimento de todo o time e garantir que os sprints sejam realmente executados.

* Stakeholders:

- Conjunto que reúne os partes interessadas (pessoas para quem o produto está sendo projetado.)
- Definem os requisitos do projeto.
- Fornecem feedbacks.
- Participam de testes.

* System architect:

- Define toda estrutura do software.
 - Pacotes de arquitetura.
 - Linguagem, frameworks, banco, cloud
 - Frontend, backend, APIs
- Toma as decisões técnicas críticas.
 - Define tudo que será criado.
 - Define o porque e o como.
- Garante os requisitos não funcionais.
- Define os padrões e as boas práticas.

* UX designer:

- Define como o software se comunicará com as funcionalidades com o usuário.
- Define como o usuário se comunicará com o software.

* Developer:

- Implementar a arquitetura.
- Codificar os requisitos CSRS/.
- Implementar o design definido.

* Tester / Q.A engineer:

- Criar e executar casos de teste.
- Providenciar feedbacks ao time de desenvolvimento.

* Site reliability engineer (SRE):

- Tem conhecimento amplo em TI.
- Parteam incidentes,
- Automatizam sistemas, procedimentos e processos.
- Auxilia na resolução de problemas.
- Parte mais importante: Garante que o produto seja confiável para o cliente.

* Product manager / Product owner:

- Define a visão do produto.
(visão de mercado)
- Entende a criação e o mercado
- Define métricas de resultados.
- Cargo de administração / negócios.

* Technical writer / Information developer:

- Escreve para o público não técnico.
- Manual de usuários, comunicados à imprensa, relatórios.

* Tech Lead:

- Tem base em fiscalizar a construção dos códigos da equipe em um curto prazo.
- Realiza code reviews.
- Mente o time.
- Ajuda com lógicas mais complexas.
- Quebra tarefas mais difíceis em múltiplas tarefas menores.

● Web and Cloud Development:

- Front-end: Lida com tudo o que acontece do lado do cliente. Lida com tudo que o usuário pode interagir!

- Back-end: Lida com tudo o que acontece no servidor antes que o código e os dados sejam enviados para o cliente. Lida com lógica, autenticação, banco de dados, segurança, processamento.

- Full-Stack

- Front-end + Back-end.

* HTML: Usado para criar a estrutura de um site. Linguagem de marcação.

* CSS: Usado para projetar o site e torná-lo atraente. Linguagem de estilo.

* Java Script: Linguagem de programação que junto ao HTML e CSS dão interatividade ao site.

• Responsive design:

- Significa que o design será redimensionado automaticamente para o tamanho do dispositivo que o está acessando.

• Pair programming:

- Dois pessoas codificando o mesmo tempo no mesmo ambiente.

- Existem vários tipos de programação em pares.

• CI/CD:

"Conjuntos de práticas e princípios de automação de integrações, teste e entrega de software, que pode ser implementado por diferentes ferramentas e pipelines.

- Uma das bases da cultura de DevOps!

- CI (Continuous Integration)

- Os desenvolvedores integram o código frequentemente em um repositório central através de commit.

- Quem é o objetivo do CI:

Objetivos centrais:

1. **Detectar erros cedo** Cada commit aciona builds e testes automáticos, identificando bugs logo após serem introduzidos — quando ainda são baratos de corrigir.
2. **Reduzir conflitos de integração** Ao integrar código com frequência (várias vezes ao dia), evita-se o famoso "integration hell" causado por branches muito divergentes.
3. **Garantir qualidade contínua** Testes automatizados (unitários, integração, lint, cobertura de código) rodam em cada push, mantendo o padrão de qualidade sem depender de revisão manual.
4. **Feedback rápido para o time** Desenvolvedores sabem em minutos se seu código quebrou algo, em vez de descobrir só no final do sprint.
5. **Manter o código sempre "deployável"** A main/trunk está sempre em um estado estável e potencialmente pronto para ir a produção.

• Algumas atividades rotineiras:

- **Commit**: Processo de salvar alterações do código contendo um autor (quem realizou) a data, uma mensagem (o que foi feito). É mantido e criado pelo sistema de controle de versões (git) e armazenado através de um código hash!

- **Push**: Processo de enviar um commit para o repositório principal através de gerenciador de versões (git).

- CD (Continuous Deployment) ou Delivery?
- Processo que ocorre após a validação nos testes em CI!
- CD constrói a aplicação como um **tarbo**!
(transfere os códigos em algum executável)
- Implanta tudo pronto em um ambiente.
 - Teste.
 - Staging.
 - Produção.

• Continuous Delivery:

- O sistema prepara tudo automaticamente, mas alguém precisa aprovar o deploy final!

• Continuous Deployment:

- Não existe aprovação manual, a validação para os testes e testes vai automaticamente para produção!

• Build Tools:

- Ferramentas que ajudam na hora de construir o software com um teste!
- Um software é um conjunto de várias partes menores, de código, linguagem, classes, ...

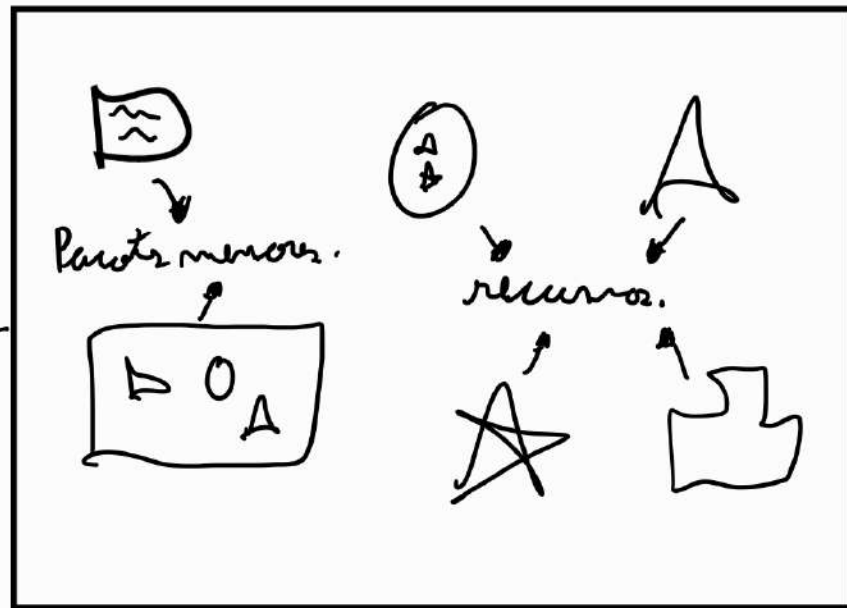
- Build é o processo de transformar o código em algo executável!
(Build $\neq$ deploy)

• Exemplos de ferramentas Build:

- Apache Maven
 - Gradle
 - Makepack
 - Vite
 - Make
- Os ajudam a tornar toda a complexidade de divisibilidade e interação muito mais fácil para tornar nossas ideias executáveis.

• Packages:

- Um pacote comumente é um pacote de outros menores pacotes!



• Packages armazenam toda a aplicação em um único lugar!, desta maneira fica muito mais prática baixar e instalar uma aplicação!

- Um pacote contém:

- App bundle.
- Instructions for installation.
- Metadata.

• Package managers:

- Os gerenciadores de pacotes cuidam das tarefas de localização, instalação, manutenção e desinstalação de pacotes!
- Gerenciam a extração e extração dos arquivos.
- Verificam a integridade do pacote, verificam os seus certificados digitais e verificam a autenticidade do pacote.

• Exemplos:

- DPKG, - RPM, - Unrelativ, - MacPorts

• Overlappation package managers:

- Quando houver uma biblioteca ou alguma ferramenta ela é instalada em novo projeto via package manager local!

- Exemplos de managers package cloud:
- Libraries and utility code are managed with the cloud application package managers

Node.js / JavaScript	Java	Ruby	Python
npm	Gradle	RubyGems	Pip
	Maven		Conda

↳ Parte muito importante no dia a dia!

• Software Stacks:

"Conjunto de tecnologias usadas juntas para desenvolver e executar um sistema em produção"

- Antes de iniciarmos o desenvolvimento de qualquer aplicação, devemos primeiro definir o que será criado e com o que será criado, desta maneira podemos identificar a nova aplicação.

Estrutura típica de um Software Stack

Um stack geralmente possui camadas como:

1. Sistema Operacional

Exemplo: Linux, Windows

2. Servidor Web

Responsável por responder requisições HTTP.

Exemplo: Apache HTTP Server, Nginx

3. Banco de Dados

Onde os dados da aplicação ficam armazenados.

Exemplo: MySQL, PostgreSQL

4. Linguagem de Programação / Backend

Onde a lógica do sistema é implementada.

Exemplo: PHP, Python, Node.js

5. Frontend

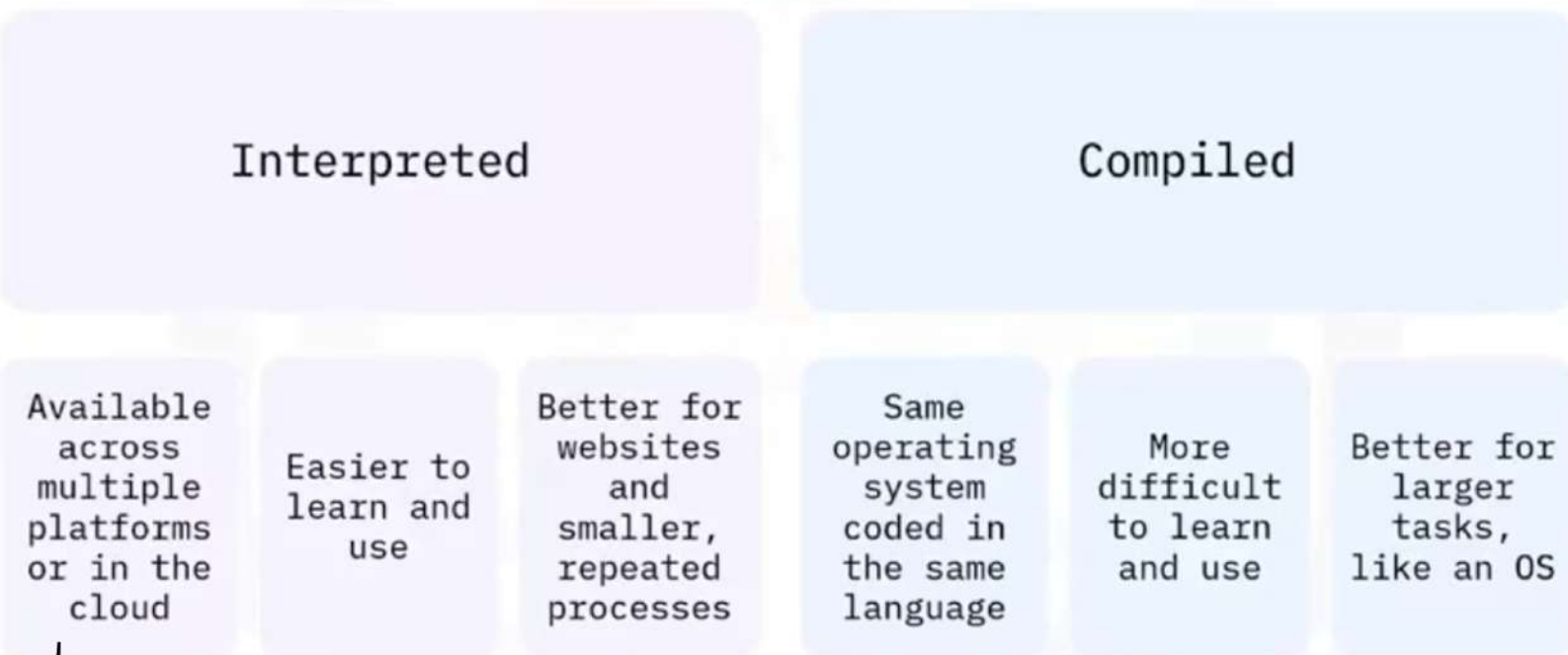
Interface que o usuário vê.

Exemplo: React, Angular

Dependendo da aplicação a estrutura do Stack pode ser diferente ou não!

• Principais diferenças entre uma linguagem interpretada e uma compilada:

Interpreted versus compiled



↳ Existem também outras linguagens que exibem de outros berrros, porém elas não são as principais!

• Existem vários tipos de linguagem:

- Alto nível: Mais próxima da linguagem humana. Exemplos: Python, Java, C, C++, C#. Eles tem menor controle direto do hardware! Não precisam lidar diretamente com ele! São como uma ponte entre humanos e máquina.
- Baixo nível: Mais próximas da linguagem de máquina! São próximas do hardware do computador, compreendem apenas 0s e 1s! Exemplos: MOV, ADD e AX, existem muitas linguagens de baixo nível, pois, estão muito ligadas ao hardware! E existem muitos diferentes hardwares!

- Query (Pesquisa):

- Linguagem de query não usada para consultas, busca e manipular banco de dados! São de alto nível! Escrita de ler e escrever! Formato CRUD.

Exemplos da principal linguagem SQL.

`</> SQL`

```
SELECT nome  
FROM alunos  
WHERE idade > 18;
```

Fácil execução?
(alto nível)

Quem "compila" e executa este código é o SGBD (Sistema Gerenciador de Banco de Dados).

• Arquitetura de software:

"organização do sistema".

- Uma arquitetura define como os componentes devem interagir entre si, o ambiente operacional e as principais regras para projetar o software.

- A arquitetura é muito importante para antecipar os problemas! Um bom arquiteto consegue prever quais serão as principais dificuldades de implementação de software e de avaliar as possibilidades dos stakeholders!

- Um arquiteto cria da:

- SDD (Software Design Document).

- Architectural diagrams.

- UML diagrams.

- Orienta os desenvolvedores sobre o ambiente no qual o software é lançado.

• Padrões de Arquitetura (muito importante!)

"Formas conhecidas e reutilizáveis de organizar a estrutura de um sistema de software"

(Architectural Pattern $\neq$ Design Pattern)

• Exemplos dos principais padrões:

1 - Layered Architecture.

↳ Sistema é dividido em camadas, cada uma com a sua responsabilidade!

2 - MVC (Model-View-Controller).

Model = Dados e regras, View = Interface gráfica, Controller = lógica

3 - Microservices

↳ Sistema é dividido em vários serviços pequenos e independentes.

4 - Event-Driven

↳ Componentes se comunicam através de eventos.

5 - Existem muitos outros!

• Cloud deployment types:

- Public:

- vários stakeholders dividem o mesmo servidor (AWS, AZURE, Google cloud).

- Private:

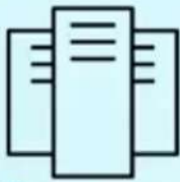
- "Servidor dedicado", servidor(s) para um único stakeholder.

- Hybrid:

- Ambos Public e private trabalham juntos cada um com as suas vantagens!

• Mais sobre UML:

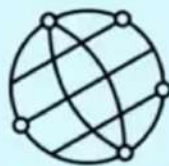
Web and Application Servers



Servers

Provide services, resources, data, or applications to a client

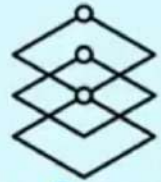
Store, process, and manage network data, devices, and systems



Web Servers

Delivers web pages, files, images, and videos to the client

Responds to HTTP requests from a browser



App Servers

Runs the apps and provides the app to the client or the web server

Stores code that determines how data can be created, stored, or changed

• Proxy Server:

- Servidor intermediário que lida com as solicitações entre dois computadores.

- Tem várias finalidades:

- Balanceamento de carga.
- Atenuação do sistema.
- Armazenamento em cache.
- Atenuação como firewall.
- Criptografia.
- É muito mais (realmente da, para fazer muita coisa com proxy).

- Skills Required for Software Engineering.

What are hard skills?

- Programming languages
- Version control
- Cloud computing
- Testing and debugging
- Monitoring
- Troubleshooting
- Agile development
- Database architecture

↳ Habilidades obrigatórias para estar diretamente relacionado ao desenvolvimento cotidiano de um software.

↳ o principal: Resolver problemas!

• Carreiras na Engenharia de Software:

- Carreiras mais ligadas aos desenvolvimento:

- Estagiário

- Desenvolvedor Júnior,

- Desenvolvedor Pleno,

- Desenvolvedor Sênior.

- Carreiras mais ligadas a gestão:

- Tech Lead

- Arquitetos de Software,

- Engineering Managers.

- Diretores de Engenharia.

- CTO (Chief Technology Officer).

(algumas vezes, podem atuar de empresa para empresa, mas normalmente as funções são extremamente parecidas)

- Trabalhos dentro da engenharia de software:
 - Front-end:
 - Back-end:
 - Full-stack:
 - DevOps:
 - QA:
 - Integrations engineer:
 - Security:
 - Mobile:

• Princípios de código de ética:

- São 8 princípios:

1º Public

2º Client / Employees

3º Product

4º Technology

5º Management

6º Profession

7º Colleagues

8º Self

